Project Part II — The Chess Opening Trainer (COT)

April 17, 2019

1 Overview of the implementation

In this section we will discuss the main idea behind the COT both intuitively, as in this project's first part, as well as thoroughly, through an inspection of the application's code.

1.1 A quick reminder: What is the COT?

In this first section we will provide a brief reminder of the main ideas behind the COT. As it has already been discussed, the main idea behind the COT is to create an application that takes advantage of the already stored human knowledge about chess and, more precisely, chess opening theory, condensating this knowledge in the form of rules regarding the strategy to be followed. To do so, the main idea is to let the application first enter a supervised training loop in which it will "learn" an approximation of a function defined above the cross product $\mathcal{R} \times \{1, 2, \ldots, 10\}$, where with $\mathcal{R}$ we denote the (finite) set of all the rules used. To do so, the application processes the training examples provided and examines whether each rule is followed or not and updates the values of a $10 \times n$ matrix — where n is the number of the rules used in our implementation. At the end of the procedure, the values of the matrix are normalized to the interval $[0, 1]$, so, one may consider them as probabilities of each rule being followed during move[1] i, $i = 1, 2, \ldots, 10$.

Before we proceed, we would like to mention that, instead of following a two layer application, as described in the first part of this project, separating rules into *objective* and *subjective*, we insisted on considering all the rules as a unified set, since the former approach did not seem to offer the wanted results, while it was computationally costful. We will come back to this issue and other future expansions of this application in the last section.

[1] We remind here that we consider the opening of a chess game to last, roughly, for 10 moves.

1.2 Inspection of the application's code

Here we will deal with more detail with the majority of the application's parts. To begin with, the application has been developed using Java 8 and it includes five classes: `Chessboard, Move, Example, Agent` and the `Test` class, which includes that `main` method.

1.2.1 The `Chessboard` class

The `Chessboard` class is the main class of the application (lines 45-685), in which all the elementary features of a chess game are implemented. More specifically:

- It includes a character 8×8 array, `board`, which is the chessboard itself, including character symbols for all its components; white pieces, black pieces and blank squares. More specifically,
  - capital letters are used for white pieces;
  - lower-case letters are used for black pieces;
  - the standard algebraic notation is used for each piece ("R" for rook, "N" for knight, "B" for bishop, "K" for king and "Q" for queen), with the extra convention that "P" represents a pawn — we will refer to this notation as *intermediate* notation, as it functions as a stepping stone between the usual algebraic notation and a full notation of the form piece-origin-target;
  - a dot character "." is used to represent blank squares.
- It includes six (6) boolean parameters (lines 47-52), which are used to verify whether there has been made any move that violates the castling restrictions (neither rook nor king has moved from their initial position).
- It includes two construcors; one that takes an 8×8 array and initializes the board field of the class to that array and the usual void constructor (lines 54-62).
- It includes a `printBoard()` method, which prints the `board` array at its current state and is used both for interaction with the user and for debugging puproses (lines 64-74).
- It includes an `initBoard()` method, which initializes the board field to the usual starting setting of a chess game (lines 76-104).
- It includes a `getSquare(x,y)` method, which takes two integers and returns the piece contained in that square on the board (lines 106-108).

- It includes two methods (lines 110-116), which verify whether two pieces are of the same colour or not.
- It includes a `inBoard()` method, which verifies that a square is within the boards limits (lines 118-120).
- It includes a `containsMove(list,move)` method, which takes a arraylist of moves and a specific move and checks whether the latter is included in the list (lines 122-133).
- It includes six (6) methods (lines 135-348), each one finding the legal moves a piece is able to make at each specific time of the game. All these methods return an arraylist containing all these moves.
- It includes a `castle(c)` method, which executes a castle move, if the prerequisites are met (lines 350-368).
- It includes a `castleCheck(c,i,j)` method, which verifies whether a castle move can be done and is used later on to update the castling availability for each side (lines 371-390).
- It includes two methods (lines 392-470), which are used to return the legal moves that can eb made by the black and the white player respectively, making use of the other methods that check for legal moves on the board.
- It includes a `legalMoves()` method, which returns an arraylist of all the legal moves available on the chessboard (lines 472-479).
- It includes an `undoMove(move)` method, which takes a move and returns the chessboard to its state before that move had been executed (lines 481-516).
- It includes a `executeMove(move)` method, which takes a move expressed in usual algebraic notation and executes that move editing appropriately the `board` field. Also, for reasons that will be discussed later on, it returns the move as an object of the `Move` class (see next). (lines 518-679).

1.2.2 The `Move` class

In this class (lines 682-760), we represent any move that may be executed on the chessboard, encoded in a format that makes available the restoration of the chessboard's previous state, as well (see `undoMove(move)` method in the `Chessboard` class). More specifically:

- It includes a character field, `piece`, which represents the piece itself and four integer fields (lines 683-687), which represent the initial (`iRow, iCol`) and final (`fRow, fCol`) positions of the piece moved.

- It includes three constructors (lines 689-705); one that takes the values of the five fields defined as above and initializes them, a copy constructor which copies another `Move` object in the new one and the usual void constuctor.
- It includes "getters" for all its fields (lines 707-724).
- It includes a `printMove()` method, which prints the five (5) fields and is used mainly for debugging (lines 726-732).
- It includes a `toString()` method that over-writes the `Object` class `toSrting()` one and which is used to express the current move into a semi-algebraic notation (lines 734-755).
- It includes an `equals(move)` method, which is used to verify that two moves are, actually, the same one (lines 757-760).

1.2.3 The `Example` class

In this class (lines 762-824), we represent an arbitrary training example that will be used during the agent's training phase. More specifically:

- It includes a String 10×2 array field, `table`, which is used to store the 20, in total, moves of every training example — we remind here that COT is trained to play the first 10 moves on the black side (line 763).
- It includes s single constructor (lines 765-767), which takes a file to initialize the `table` field.
- It includes a `readExample(file)` method, which takes a file and updates the `table` field with the moves included in this file. Note that the file should include the moves in algebraic notation and, more specifically in the format "nn. Move1 Move2", so as to be parsed. For instance, the format "12. Nf3 bxc4" is accepted, while "12.Nf3 bxc4" is not recognized. Also, any other symbol, such as checkmate (#) or check (+) should be omitted[2] (lines 769-810).
- It includes a `printExample()` method, which is primarily used for debugging purposes (lines 812-819).
- It includes a `getMove(i,j)` method, which returns the move located on some specific position of the `table` field (lines 821-823).

[2] As for the chekmate case, it is not expected that a training example will come from a game that ends prior to move 10.

1.2.4 The `Agent` class

The final class of the application (lines 826-1272), which is used to represent the agent learning how to play chess during the opening phase of the game, integrating features from all the above classes. More specifically:

- It includes an integer field `n`, which is equal to the numberof the rules used. In this implementation, it is fixed to 15 (line 827).
- It includes a double 10×15 array, `firstLayer`, which is the array containing all the statistical information about the studied examples (line 828).
- It includes a `Chessboard` type object, `board`, which is used to assist studying a raining example (line 829).
- It includes a single construcor (lines 831-833), which takes an integer `n`, which corresponds to the rules' number.
- It includes an `initFirstLayer()` method, which initializes the `firstLayer` field (lines 835-841).
- It includes a `standardizeFirstLayer()`, method, which is used to standardize the `firstLayer`'s values into $[0, 1]$ (lines 843-854).
- It includes a set of 15 rule methods (lines 856-1081), which are all used to represent the rules upon which the agent decides to make its next move. For more about each rule, see the comments next to each function's declaration.
- It includes a `study(e)` method, which takes an example and loops until it has processed it all, updating the values of the `firstLayer` field with the number of times each rule appears in the provided example per move (lines 1083-1144).
- It includes a `play(move)` method, which takes a chessboard, a move and he previously played move and chooses, according to the values found in `firstLayer`, which is the move that should be played next by the black player. To do so, it starts by checking the most frequent rule and then prunes the arraylist containing all legal moves so as to keep only these agreeing with that rule. Then, it continues processing that way with the second most frequent rule for that move and loops until either all the rules are exhausted, or until some rule leaves no move available. In the first case, it chooses randomly from the remaining moves, while in the second, it first backtracks so as to exclude the last rule and, again, chooses randomly one of the remaning moves (lines 1146-1248).

- It includes a `maxMove(array)` method, which returns the maximum move of a `Move` type array (lines 1250-1261).

- It includes a `train(directory)` method, which takes a directory path which contains all the training examples and, using the above methods, lets the agent study the examples and, finally, standardizes the results (lines 1263-1272).

2 Evaluation of the application

In order to evaluate our application, we can study a sample game our agent plays against white, having been trained on the provided training set of 104 examples. The game is demonstrated in figure 1.

As it can be seen COT has a pretty much normal behaviour during the first four (4) moves, until, it decides to move a piece from a more powerful position to a weaker one, closing many of its lines (5... Nce7?). After that, another inexplainable move follows, (6... b6) and then, COT proceeds normally for the rest four moves, continuing its pieces' development and, possibly, preparing a castling move.

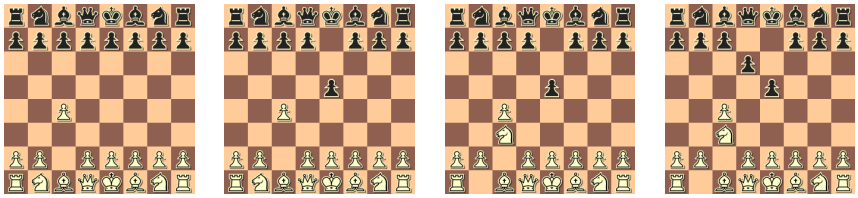
One could account for these inexplainable moves and general COT's instability[3] based on two facts:

1. COT has been trained in a relatively small number of examples (104), so, it is possible that there is sill some high level of inaccuracy in its actions due to not having finely tuned its weights in the `firstLayer` of `Agent` class. Thus, results from further training on more examples would shed a light on COT's unstable behaviour.

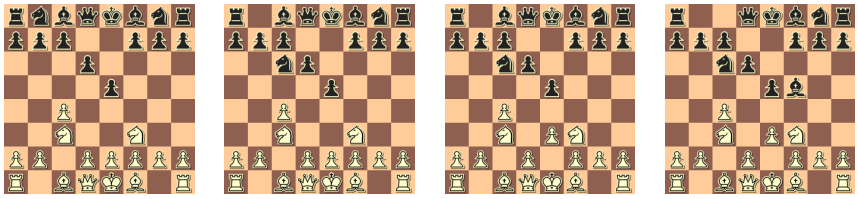
2. The rules used are either not enough to capture the entire complexity of a chess opening after some certain number of moves (in other cases, such unstable behaviour appears around movve number 5, as well) or they are too generally expressed to encapsulate the game's complexity. It may be wiser to use non-boolean rules (e.g. rules that return fuzzy values, as suggested in the next section), so as to have a deeper insight of the situation on the board and the actions worth cosnidering.

Taking the above into consioderation leads us to the conclusion that COT's implementation is, up to some point, sucessful, since it has suceeded in playing some moves included in the main line of several classical openings (see [1] for a usual line similar to that of figure 1 for the English game). Howeverm, there is a lot to be done before we are able to claim that COT is capable of playing chess close to an intermediate player's level.

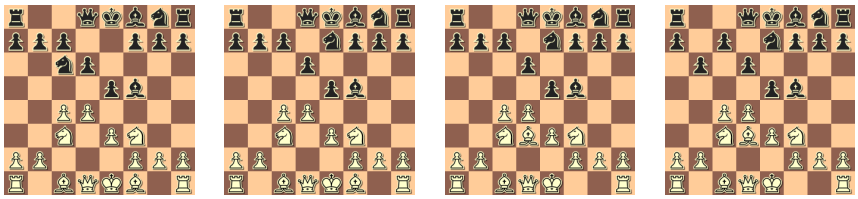
[3]Similar patterns of inexplainable moves have been noted in other games played with it, as well.



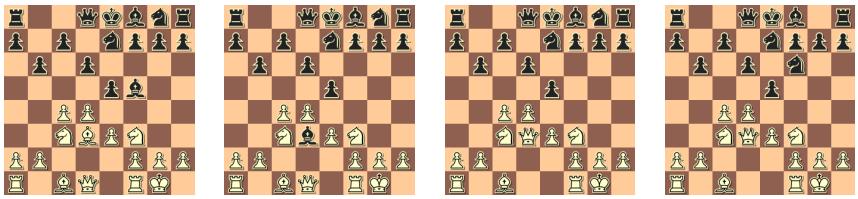
(a) 1. c4 (b) 1. . . . e5 (c) 2. Nc3 (d) 1. . . . d6



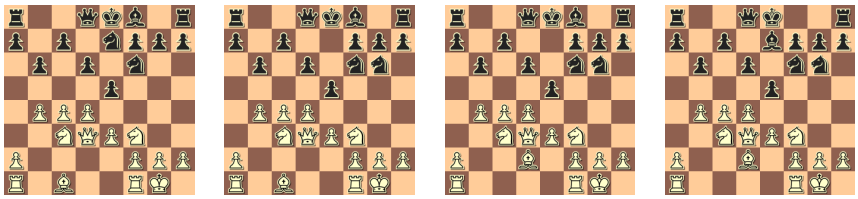
(e) 3. Nf3 (f) 3. . . . Nc6 (g) 4. e3 (h) 4. . . . Bf5



(i) 5. d4 (j) 5. . . . Nce7? (k) 6. Bd3 (l) 6. . . . b6?



(m) 7. O-O (n) 7. . . . Bxd3?! (o) 8. Qxd3 (p) 8. . . . Nf6



(q) 9. b4 (r) 9. . . . Ng6 (s) 10. Bd2 (t) 10. . . . Be7

Figure 1: A sample game with COT.

3 Possible future expansions

In this section, we will discuss some ideas regarding COT's possible future expansions.

3.1 Non-linear dependence on rules

One way to expand COT has already been discussed in the first part of this project; instead of searching and evaluating single rule activations, one may examine all the pairs of rules that are followed together. This information would be stored in a $10 \times n^2$ array, which would, of course, increase the time complexity of the training procedure, but it would also, possibly, provide a deeper insight in the ways moves are executed by our agent and, thus, result in a more reliable way of playing, for a longer interval theoughout a chess game (e.g. more than 10 or 12 moves). The same idea, about checking not sungle rules, but sets of rules, could be extended to include $k-$sets of rules, which would, however, rise time complexity to n^k, and, if one decides to check all possible rule subsets so as to draw a conclusion, the time complexity would become 2^n, since the subsets of a set of n elements are exactly 2^n. The latter, however, seems as a computationally unrealistic approach.

3.2 Fuzzification of rules

Another option, unexplored in the first part of this project, has to do with the way the rules are expressed and assessed. As one may easily verify, in our current approach, all the rules are treated as boolean functions which, given a state of the board and/or a specific move, make a judgement about whether the specific rule is followed or not and then, the agent is to decide how to interpret this piece of data. However, simplistic this approach may seem, the agent is capable of playing at a novice level for a fair amount of moves. Nevertheless, the above implementation of the rules' concept, unveils a belief/missconception about their nature; that is, in all the above, the rules are treated as if they are crisp (binary) entities that are either entirely true or entirely false. What if, some rules are true up to some level but neither absolutely true or absolutely false?

Consider, for instance, the rule concerned with center influence. We have stated that this rule should be considered to be followed whenever a piece moves to a square from where it can then control a square located on the center of the chessboard. Assume, now, that we are playing with the white. Then, both Nf3 and Ne2 provide us with some control over the chessboard's center — the first move enhances our control on d4 and e5, while the second one enhances our control only on d4. However, Nf3 is way more often preferred as the king's knight first move, when compared to Ne2, exactly due to the fact that it provides the white player with a wider influence of the center. That is, even if both moves are instances in which the "Control center!" rule is followed, one of them seems to serve the rule's purpose more adrequstely. Given that, we realize that, in fact, rules provide us with more insight and strategical depth, if they are considered as fuzzy entities.

To implement the idea above, one could allow the rules take values on the interval $[0, 1]$, so as to "fuzzify" them. However, which should be the optional activation level for each rule, at each move? One way to approach this issue is to let COT learn that level for each rule and each move, by expanding the `study(e)` method as follows. Instead of investigating whether each rule is followed at each move, one could, for the rules that can be fuzzified, also note the relative frequency with which each activation level of a rule appears per move. For instance, as far as the "Control center!" rule is concerned, there are 4 discrete levels of influence (controlling, 1, 2, 3 or all 4 central squares), so, in this case, we should also take note of the activation level per move. For this purpose, we could use a separate `Rule` object class, having as one of its fields an array, in which the various relative frequencies of each activation level are stored.

3.3 Opening types

Another fact that seems to affect the way rules are applied is the type of the opening; that is, whether it is an open one (e.g. Ruy Lopez), a semi-open one (e.g. Sicilian defence) or a closed one (e.g. Queen's gambit, declined). For instance, in an open game, one may need to castle early on to ensure their king's safety, while, there are several examples of more closed games where there is no safer place for the king than the center (e.g. in the Nimzo-indian defence, with a totally pawn-closed center). So, whether a rule is followed or not is also a matter of the opening type of the game.

To integrate this fact into our application, seems and is a hard task. One possible idea would be to first preprocess the training dataset and detect three (or as many there might occur) clusters of opening types, based, fir instance, on the number of available legal moves for each side (or, even, in total). Thus, we would have to break the learning procedure into three stages, one for each separate cluster and then, during the game, evaluate in an "on-line" manner, which type of game (or which convex combination of them) our agent should stick to.

Final remarks

For the selection of the rules, [2,3,4] were used, while the training examples were constructed based on [5,6].

References

[1] `http://portablegamenotation.com/ECO.html`, retrieved on 29/03/2019.

[2] Lasker, E. (1945), *Common Sense in Chess*, David McKay Company, Inc., New York.

[3] Siaperas, T. (1967), *Chess, Vol. I*, Aposperitis Publications, Athens.

[4] Siaperas, T. (1977), *Chess, Vol. II*, Aposperitis Publications, Athens.

[5] Grivas, E. (2005), *Immortals: FIDE's World Championships, 1886-2005*, Kedros Publications.

[6] `http://www.chessgames.com/index.html`, retrieved on 29/03/2019.